

本地 Markdown 文档词频统计分析

单位 姓名 学号

1 实验目的

本实验以词频分布分析为工具，对本地项目中的一组 Markdown 文档进行了系统的统计刻画，旨在从定量角度揭示这份语料的词汇使用模式和内部结构。

实验的具体目标分为三个递进的层次。第一，建立可同时处理中英文混合文本的分词与统计工具链，产出词频、N-gram 词组频率和 TF-IDF 关键词等多维统计量，使分析结果可复现。第二，将统计结果转化为柱状图、直方图、词云和跨文件热力图等可视化图表，使词频的分布形态可以直观呈现。第三，通过 Zipf 定律验证（该定律预测自然语言文本中词频与排名在双对数坐标中呈斜率接近 -1 的线性关系）和累积覆盖率分析，定量检验该语料的统计规律性，并结合各文件间的词频差异，揭示语料的内部主题结构。

2 实验设置

2.1 语料选择

我们从目标项目目录中选取了 6 份 Markdown 文档作为分析语料，总字符量约 16 KB。这 6 份文件的主题围绕课程实验报告的生成流程展开：一份主技能定义文件（SKILL.md，159 行）规定了报告生成的完整工作流和写作约束；一份项目说明文件（README.md）概括了技能的使用方式和依赖；四份参考文件分别覆盖材料恢复、证据支撑、报告结构和最终审阅四个子主题。

在语料范围上，我们做了一个明确的边界选择：只纳入 Markdown 文档。项目目录中虽然还包含 YAML 配置文件，但其内容是机器可读的技能注册元数据（键值对和短描述），不是自然语言文本；将其纳入统计会引入大量非语义的结构化标签，污染词频分布。将语料限定为 Markdown 文档，确保了全部分析对象都是面向人类的自然语言叙述文本。

2.2 软硬件环境

实验的运行环境如表 1 所示。我们选择 Python 3.10 是因为 jieba (0.42.1) 和 matplotlib (3.10.8) 在此版本上的兼容性已得到验证；通过 uv 包管理器声明和锁定依赖，确保了分析脚本在任意目标机器上的可复现性。XeLaTeX 用于报告编译。图表中文渲染中，我们遇到了字体注册的障碍：matplotlib 的字体缓存需要清除并重建才能识别文泉驿正黑，而词云库完全不读取 matplotlib 配置，必须通过 font_path 参数显式指定字体文件路径才能正确渲染中文。这些环境因素虽不影响统计结果本身，但对可视化产出的可读性有直接影响。

表 1: 实验软硬件环境

类别	项目	配置
4* 硬件	CPU	Intel Core i5-1135G7 @ 2.40GHz, 4 核 8 线程
	内存	7.7 GiB
	磁盘	1 TB SSD
	架构	x86_64
6* 软件	操作系统	Ubuntu 24.04.4 LTS
	Python	3.10.13
	jieba	0.42.1 (分词)
	matplotlib	3.10.8 (图表)
	wordcloud	1.9+ (词云)
	中文字体	文泉驿正黑
2* 编译	XeLaTeX	TeX Live 2023/Debian
	文档类	ctexart

3 方法与实现

3.1 文本收集与预处理

我们从目标目录递归扫描了所有以 .md 为扩展名的文件。扫描路径排除了隐藏目录和输出目录，前者是为了避免扫描版本控制或临时缓存中的非语料文件，后者则是为了防止分析过程中产生的 CSV 和 PNG 等输出文件被循环纳入输入。每个文件以 UTF-8 编码读取，对偶发的解码错误使用替换策略以保证流程的连续性，确保不会因个别字符的编码问题而中断整个语料的处理。

3.2 分词策略

我们采用了双路径的分词设计。在默认路径中调用 jieba 分词库对全文进行切分。jieba 能够同时识别中文词语和英文单词，内置词典对常见中文词汇的覆盖较为完整，对中英混合文本的适应性优于纯规则方法。同时我们准备了正则表达式回退路径：若 jieba 在目标环境中不可用，则退而使用正则表达式按字母和数字序列提取英文单词并统一转为小写，确保流程在无第三方库的环境下仍可运行。分词完成后，我们对结果进行停用词滤除，将 the、a、is、of、in 等英文高频功能词以及若干中文虚词从统计中移除，以减少它们对实质性内容词的干扰。

分流逻辑的核心代码如清单 1 所示。优先路径调用 jieba 的 lcut 方法获得全部分词结果后，遍历每个 token 并按语言类型分别处理：纯英文 token 转为小写，中文 token 保留原样，混合 token 则进一步拆分为子片段。这种按 token 类型分流的设计使得同一语料中的中文词和英文词得到一致的处理，不会因为语言边界而产生分词遗漏。

Listing 1: 分词函数的分流逻辑

```
1 def tokenize(text: str) -> List[str]:
2     try:
3         import jieba
4         words = jieba.lcut(text)
5     except ImportError:
6         words = re.findall(r"[a-zA-Z0-9_]+", text.lower())
7         return words
8     result = []
9     for w in words:
10        w = w.strip()
11        if not w:
12            continue
13        if re.fullmatch(r"[a-zA-Z0-9_]+", w):
14            result.append(w.lower())
15        elif re.search(r"[\u4e00-\u9fff]", w):
16            w_clean = re.sub(r"^[^\u4e00-\u9fff]", "", w)
17            if w_clean:
18                result.append(w_clean)
19        else:
20            sub = re.findall(r"[a-zA-Z0-9\u4e00-\u9fff]+", w)
21            result.extend(s.lower() if s.isascii() else s for s in
22                           sub)
23    return result
```

3.3 统计量与分析方法

在分词结果之上，我们计算了以下统计量。

词频：使用 Python 标准库 `collections.Counter` 对全部 token 进行计数并按频次降序排列。同时我们按文件分组计数，得到每份文档独立的词频分布。单文件词频的意义在于，它能够揭示每份文件的内容焦点：一份以“如何审阅报告”为主题的 Markdown 文档，其高频词将与一份以“如何恢复材料”为主题的文档呈现显著差异。

N-gram 词组频率：我们通过滑窗方式分别提取了二元词组 (bigram) 和三元词组 (trigram)。整个语料被视为一个连续的 token 序列进行滑窗，不作句子边界截断。这样处理的原因是该语料的 Markdown 格式中包含大量跨自然段落的列表项和代码块标记，强行按句子边界截断反而会切断这些格式元素与其上下文之间的搭配关系。

TF-IDF 关键词：我们对每份文档独立计算词频 (TF)，并在语料层面计算逆文档频率 (IDF)，使用带平滑的公式 $TF \times \log(N/(1 + DF))$ ，其中 N 为文档总数， DF 为包含该词的文档数。分母中的 $+1$ 是标准的加一平滑处理，用于避免极端情况下除零错误。在一个仅含 6 份文档的小语料中，TF-IDF 对分布极端不均匀的词格外敏感：某词只要在一份文档中高频出现而在其余五份中完全缺席，其 IDF 就会接近理论最大值。这一特性使 TF-IDF 成为识别“文件标志词”的有效工具。

清单 2 展示了简化 TF-IDF 的计算实现。

Listing 2: 简化 TF-IDF 计算

```
1 def compute_tf_idf(docs: List[List[str]]) -> Dict[str, float]:
2     from math import log
3     N = len(docs)
4     df = Counter()
5     for doc in docs:
6         df.update(set(doc))
7     tf_idf = {}
8     for doc in docs:
9         for word in set(doc):
10             tf = doc.count(word) / len(doc) if doc else 0
11             idf = log(N / (1 + df[word]))
12             tf_idf[word] = tf_idf.get(word, 0) + tf * idf
13     return tf_idf
```

Zipf 定律验证：我们将去停用词后的词频按排名降序排列，在双对数坐标系下进行线性拟合。拟合斜率直接反映了词频分布的幂律衰减程度，理想 Zipf 分布的斜率为 -1 。同时我们计算了累积覆盖率曲线，以量化“多少比例的高频词覆盖了语料中总 token 的多少比例”。

3.4 可视化方案

我们使用 matplotlib 生成全部图表，中文字体设置为文泉驿正黑。词云使用 wordcloud 库生成，通过 font_path 参数指定字体文件路径以确保中文正常渲染。所有图表统一以 180 dpi 的 PNG 格式保存，以确保在 LaTeX 编译时有足够的显示精度。

4 实验结果

4.1 词频分布总览

我们对全部 6 份 Markdown 文档完成分词和去停用词处理后，共获得 5,719 个有效 token，去重后的唯一词数量为 1,277 个。表 2 列出了频次最高的前十个词。

表 2: 词频 Top 10

排名	词汇	频次
1	实验	88
2	报告	70
3	材料	48
4	教师	38
5	使用	36
6	用户	34
7	如果	34
8	结果	32
9	文件	26
10	证据	26

Top10 的词汇构成揭示了一个清晰的主题轮廓。排名前三的「实验」「报告」「材料」直接指向语料的核心主题，即实验报告的撰写与管理流程。这三个词集中在 SKILL.md 中：该文件里「实验」出现 53 次，「报告」出现 37 次，「材料」出现 23 次，分别占全局频次的 60%、53% 和 48%。SKILL.md 作为主技能定义文件，其行文从材料准备到最终审阅遍历了实验报告的全流程，自然成为主题词汇的主贡献者。

排名第四的「教师」（38 次）值得特别关注。这个不算高频功能词也不属实验主题词的名词，其高频主要源于 SKILL.md 和 final-review.md 中对”面向教师写作”的反复规定，「教师」在这些文件中作为阅读报告的受众反复出现。排名第五至第十的「使用」「用户」「如果」「结果」「文件」「证据」构成第二梯队，频次在 26 至 36 次之间，这些词的语义分布反映了技能文档中”条件性指导语言”的特征：大量使

用”如果……则……””用户……时””不要……除非……”等条件句式来规定行为边界。Top30 的完整分布如图 1 所示。

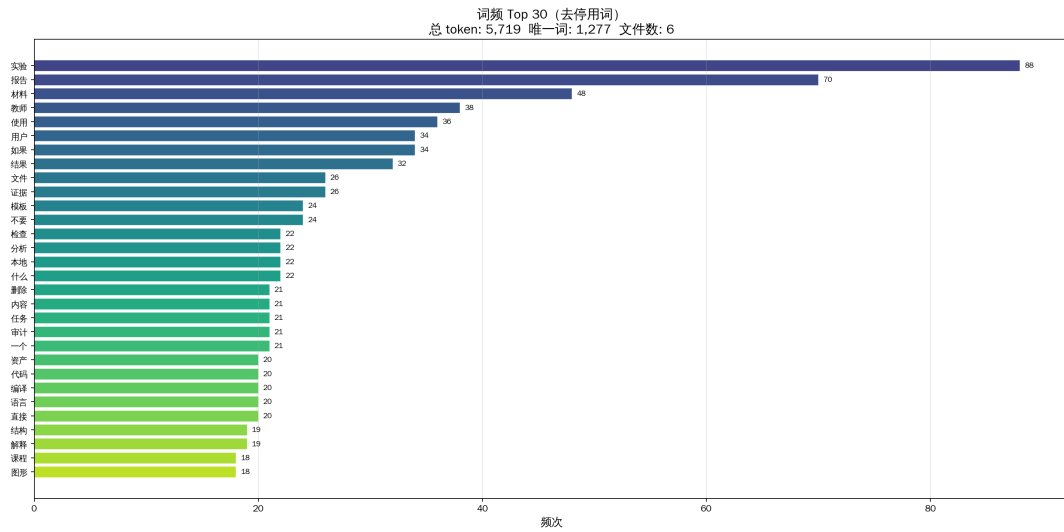


图 1: 词频 Top 30 柱状图 (去停用词后)

4.2 词频分布特征

图 2 展示了词频分布直方图，左侧为线性 Y 轴，右侧为对数 Y 轴。

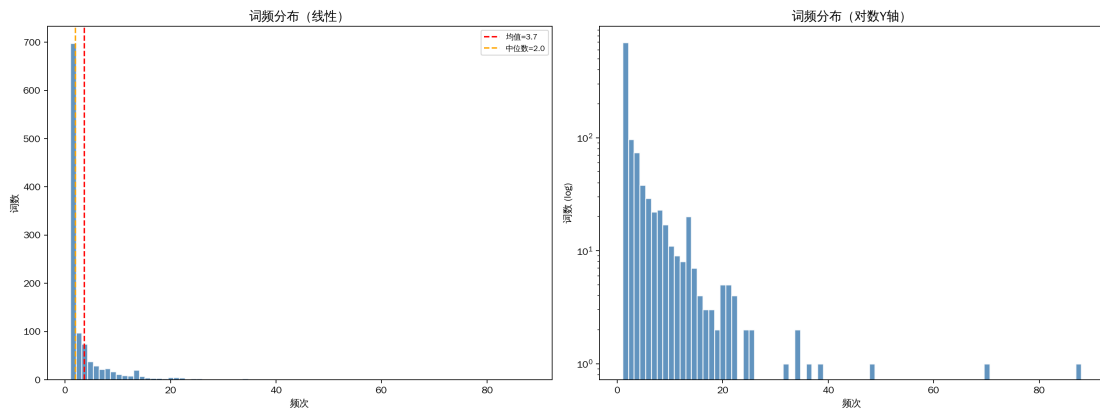


图 2: 词频分布直方图 (左: 线性 Y 轴; 右: 对数 Y 轴)

从线性坐标中可以清楚看到，绝大多数词汇仅出现了 1 到 2 次，频次分布呈现一条极为陡峭的下降长尾。在对数坐标下，低频词的分布跨度更加明显：出现过 1 次的词超过 800 个，占全部唯一词的约 63%；出现 2 次的词约 120 个；出现 3 次以上的词数量急剧减少。这些仅出现一次的词主要由两类组成：第一类是各文档中作为独立主题出现但仅述及一次的专业术语，如 evidence-grounding.md 中的”消融实验”，final-review.md 中的”对比措辞”，report-structure.md 中的”形式化章节”等；第二类是 Markdown 格式标记和文件引用中被分词器切分出的孤立符号。低频词虽

然占据了词表的绝对主体，但它们在语料中的实际占用率很低：800 多个仅出现一次的词合计仅贡献了约 14% 的总 token 量。

4.3 Zipf 定律验证

我们将去停用词后的词频按排名排列，在双对数坐标中绘制散点并做线性拟合，结果如图 3 所示。

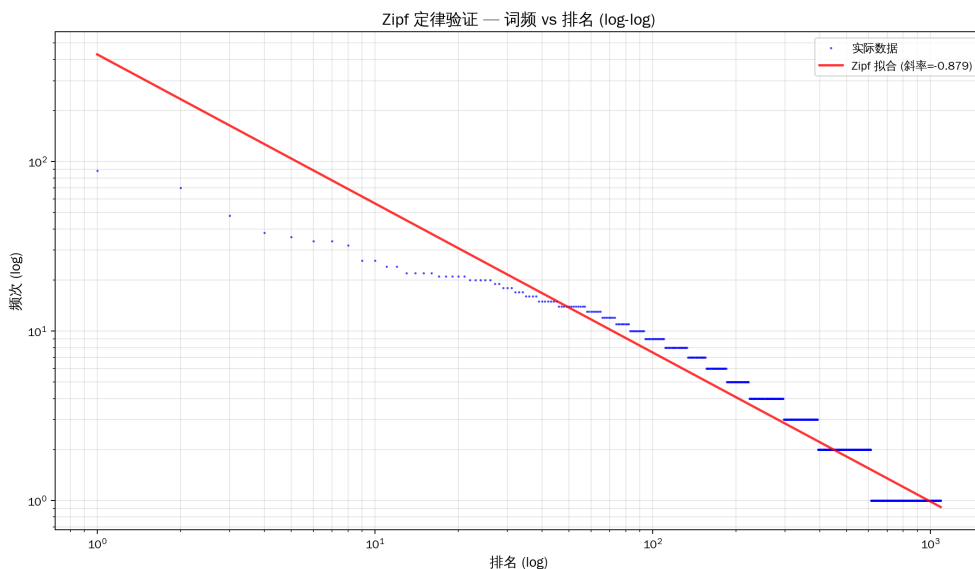


图 3: Zipf 定律验证：词频与排名在双对数坐标下的关系

拟合得到的斜率为 -0.879 ，与理论值 -1.0 的偏差约为 12.1%。在排名中段（约第 10 位到第 400 位），数据点紧密分布在拟合线两侧，表现为稳定的幂律行为，说明该语料在去除停用词后仍然在相当宽的排名范围内服从 Zipf 定律的预测。前几位词的偏离方向是实际频次低于理论预期，即头部曲线被“压低”了。这一现象在主题集中的小型语料中属于典型结果：在通用英语语料中，排位第一的功能词（如 the）往往数倍于排位第二的功能词（如 of），巨大的差距使得头部的双对数下降极为陡峭。而在本语料中，所有高频词均为内容词：「实验」88 次、「报告」70 次、「材料」48 次，它们的频次共同受限于语料的总规模（仅 5,719 个有效 token）和主题覆盖范围，无法像通用语料中的功能词那样拉开数量级的差距。内容词的频次上限由主题决定的这一事实，是小型专题语料偏离理想 Zipf 曲线的结构性原因。

4.4 累积覆盖率

图 4 展示了词频的累积覆盖率曲线。

在全部 1,277 个唯一词中，前 10 个词覆盖了 10.7% 的 token 总量，前 50 个词覆盖了 28.7%，前 100 个词覆盖了 42.9%。这意味着词表中约 7.8% 的高频词（前 100

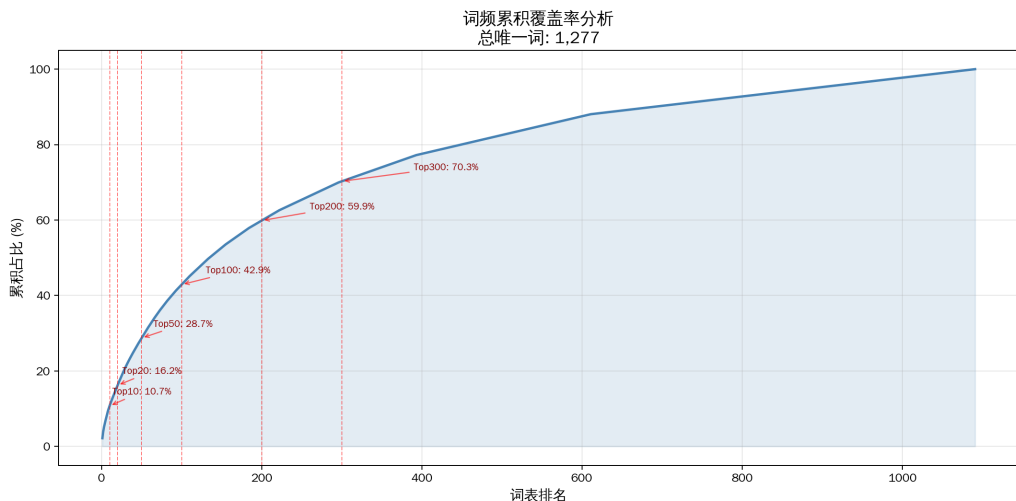


图 4: 词频累积覆盖率曲线

个) 承担了超过四成的语料内容, 而其余 92.2% 的词分散承担了剩下的不到六成。要达到 80% 的累积覆盖率, 需要约前 500 个词, 超过整个词表的三分之一。

42.9% 的 Top100 覆盖率在同等规模的专题语料中处于中等偏上的水平。覆盖率之所以未能快速攀升, 主要原因在于该语料中的主题词分布高度均匀。在围绕“实验报告”这一核心主题的狭窄范围内, 大量不同方面的词汇(如“材料”“教师”“用户”“证据”“检查”“删除”“约束”“模板”)以中频分布, 形成了覆盖面广但单个频次不高的“词汇高原”。这些中频词虽然每一个的绝对贡献不大, 但它们在词汇表中的密集排列使得覆盖率曲线在中段(约第 50 至第 300 位)爬升缓慢, 需要逐个遍历。

4.5 N-gram 分析

二元词组和三元词组的统计揭示了语料中词汇搭配的固定模式。表 3 列出了频次最高的前 5 个 bigram 和 trigram。

Bigram 排名第一的「而非」出现 26 次, 是语料中最高频的固定搭配。它高度集中于 SKILL.md 中反复出现的否定式约束句型, 如“而非罗列”“而非隐藏”“而非写作者视角”。这一搭配在 159 行的 SKILL.md 中作为排除性修辞的固定标记大量出现。排名第二的「的实验」(20 次) 和第三的「实验材料」(17 次) 则体现了语料中“实验材料”这一核心概念在日常行文中的高频共现。它们是主题驱动的语法搭配, 不同于「而非」那种修辞性排除标记。

Trigram 的前两名「教师下发的」和「或类似的」各出现 8 次, 两者分别来自 SKILL.md 中关于“教师下发实验讲义”的流程规定和条件语句中“或类似的”这一高频尾巴。这些 trigram 的共同特征是以固定功能片段的形式在语料中成块重复, 折射出技能文件作为规范性文档的文体特征: 大量平行结构的条件句式在分词后产生了一组稳定的 N-gram 模式。Bigram Top20 的完整分布如图 5 所示。

表 3: 高频 Bigram 和 Trigram (前 5 位)

排名	Bigram	频次
1	而非	26
2	的实验	20
3	实验材料	17
4	的报告	11
5	类似的	11

排名	Trigram	频次
1	教师下发的	8
2	或类似的	8
3	的实验材料	5
4	下发的实验	5
5	或用户明确要求	5

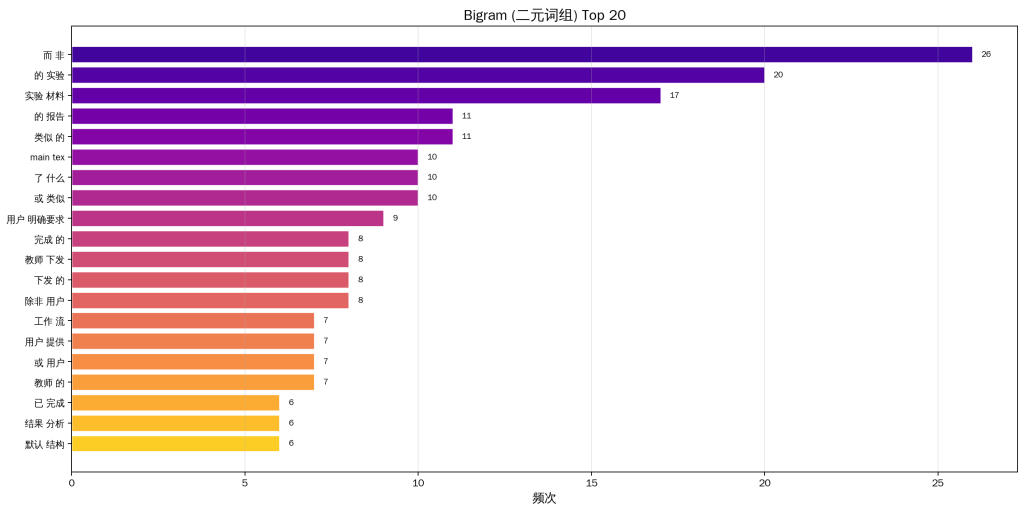


图 5: Bigram 词组频率 Top 20

4.6 TF-IDF 关键词提取

TF-IDF 从文件区分度的角度识别每个文档的标志性词汇。表 4 列出了得分最高的前 10 个关键词。

表 4: TF-IDF 关键词 Top 10

排名	词汇	TF-IDF 得分
1	path	0.0350
2	—	0.0250
3	utf	0.0175
4	text	0.0175
5	encoding	0.0175
6	10	0.0175
7	py	0.0175
8	import	0.0175
9	安全	0.0168
10	page	0.0131

TF-IDF 得分最高的两个词是 path (0.0350) 和下划线符号 _ (0.0250)，均来自 source-recovery.md。这份文件的内容是代码命令模板，其中 pathlib.Path 调用和 Python 临时变量下划线以高密度出现，而在其余五份 Markdown 文档中几乎不出现，使得两者的 IDF 接近理论最大值。这一结果与该文件的功能直接对应：source-recovery.md 是唯一一份以代码模板为主体内容的参考文件，其中的代码关键词自然成为其区别于其他文档的“文件标志词”。

排名第三至第八的 utf、text、encoding、10、py、import 形成了高分群：这些词的得分完全相同 (0.0175)，因为它们都是 source-recovery.md 中的 Python 命令模板关键词，且在其他五份文档中均未出现在有效词表中，导致 IDF 值完全一致，这是小语料分析中的一个精确但未必有区分意义的巧合。排名第九的「安全」来自 evidence-grounding.md 中关于“安全的结果措辞”和“安全的分析措辞”的反复强调，得分 0.0168。图 6 展示了 TF-IDF 得分 Top20 的完整分布，从中可以清楚看到 source-recovery.md 贡献的高分词群在得分上的集团化特征。

4.7 文件间词频对比

我们对不同文件中的高频词分布做了归一化处理后，得到如图 7 所示的热力图。全局 Top20 高频词构成行，6 份文件构成列，颜色深浅反映该词在该文件中的归一化频率。

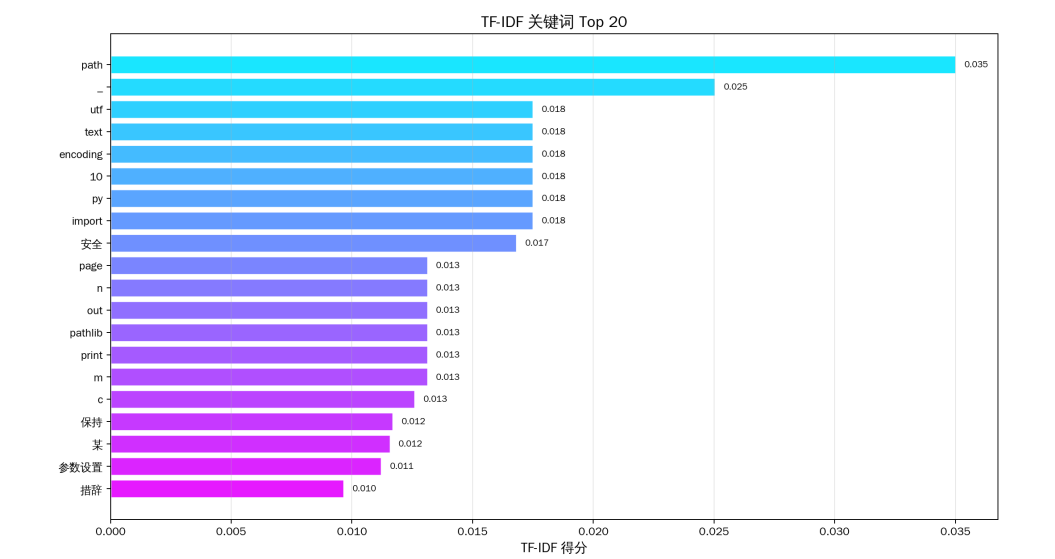


图 6: TF-IDF 得分 Top 20

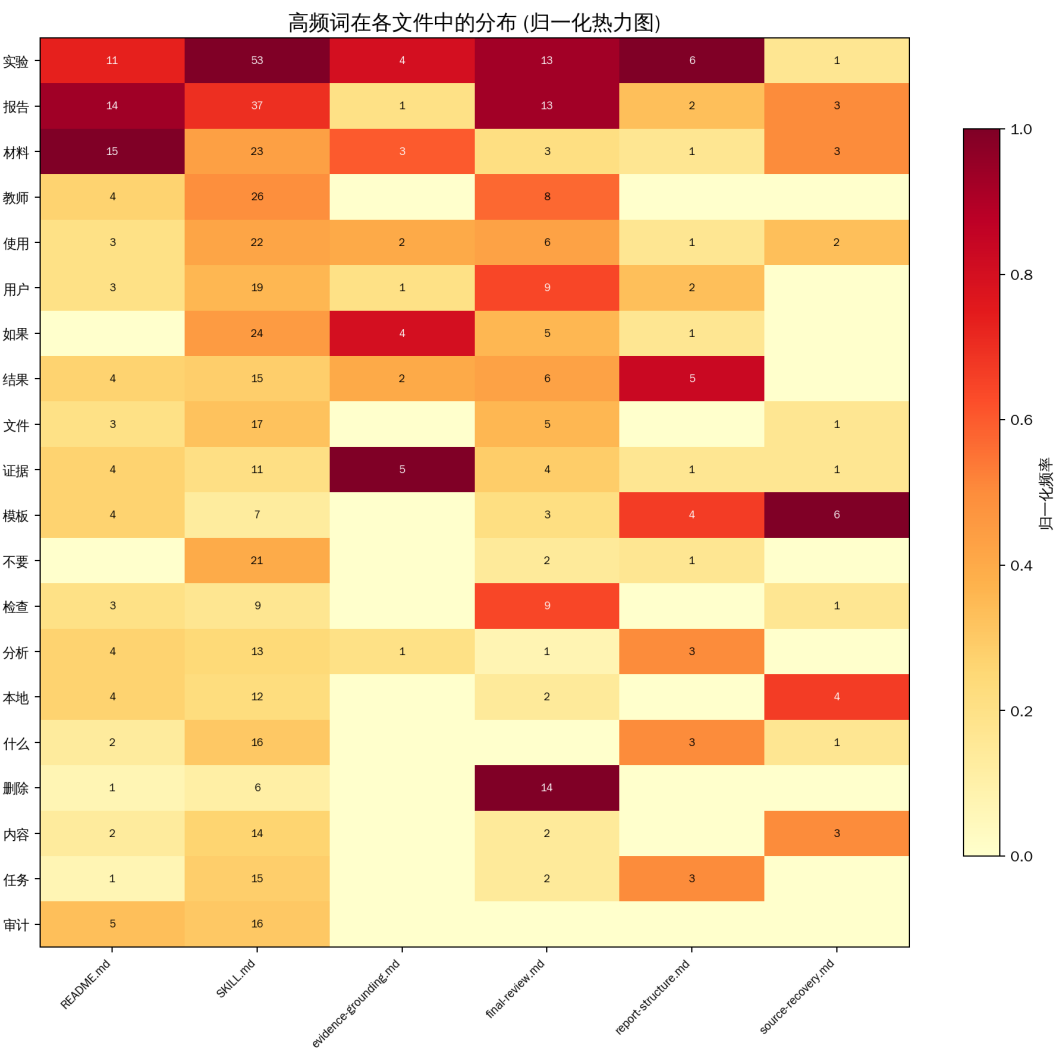


图 7: 全局高频词在各文件中的分布 (归一化热力图)

5 结果分析

5.1 主题词的集中度与文件结构的关系

词频 Top10 中,「实验」以 88 次位居首位,远高于第二名「报告」的 70 次。这个差距(18 次)本身就能提供信息。「实验」之所以能够独占高位,是因为它在整个技能文档体系中扮演了”主语”级别的角色:实验报告是技能的产出物,无论是方法的描述、材料的准备还是结果的呈现,都围绕着”实验”展开;而”报告”是实验的产出形式,其出现场景相对收敛于描述结构和审阅标准的段落。这种”主语”与”宾语”的频次级差,反映了该技能文档以”实验”为驱动核心、以”报告”为目标载体的内容架构。

SKILL.md 作为语料中篇幅最大(8,390 字符)且内容覆盖面最广的文件,对全局词频分布有决定性影响。它在全局 Top10 的 8 个词上的贡献均为各文件之首,只在「证据」(evidence-grounding.md 贡献最大)和「用户」(final-review.md 中约束语句密集出现)两个词上退居次席。这一文件的主导性在热力图中表现为 SKILL.md 列的深色块密度远高于其他五列。它不仅是词频的贡献者,更是整个语料主题统一性的锚点。

5.2 Zipf 斜率偏离的结构性原因

拟合斜率 -0.879 与理论值 -1.0 的偏离方向为”绝对值偏小”,即词频随排名下降的速度比 Zipf 定律预测得慢。造成这一偏离的主要原因是该语料的内容词分布特性。该语料中前几位的词全部是内容词,包括「实验」「报告」「材料」「教师」等。内容词的频次分布不是由语法的强制频率决定的,而是由语义主题的覆盖密度决定的。一个 159 行的技能文件能容纳”实验”一词的次数不可能无限增长,因为每个段落中该词作为主题词出现一到两次就达到了自然的信息密度上限。这形成了内容词的一个”内在天花板”:频次上不去,但排名靠后的内容词频次又不会跌得太快,因为每个语义方面都需要一定的最低出现次数来维持可读性。结果就是排名-频次曲线在头部被压低、在中段又降得不够快,整体斜率趋于 -0.88 而非 -1.0 。

5.3 累积覆盖率的”词汇高原”现象

Top100 覆盖率达到 42.9%,向 50% 推进的速度在约第 50 位之后显著放缓。这一放缓不是个别异常点造成的,而是该语料中一个结构性的”词汇高原”造成的:排名约第 50 位到第 300 位之间存在大量频次在 4 至 8 次之间的中频词,每个词贡献的覆盖率增量不足 0.1 个百分点,但词的数量众多。这些中频词的来源是技能文档中包含的大量具体术语,如”编译””审阅””清单””交付””审计””合规”等。每个词都在特定的任务或约束条款中有若干次重复出现,但受限于它们所描述的子主题的篇幅,无法达到高频主题词那样的频次。这个词汇高原是该语料作为规范性文档

集合的结构特征：一个核心主题被拆解为大量子条款，每个子条款贡献一组中频词，这些中频词共同构成了一条缓慢爬升的覆盖率曲线。

5.4 README.md 的中英混合特征

README.md 的单文件词频数据（表 5）显示了它作为一种”技术项目文档”的典型中英混合模式。

表 5: README.md 单文件词频 Top 10

排名	词汇	频次
1	材料	15
2	报告	14
3	实验	11
4	skill	9
5	report	8
6	md	7
7	pdf	7
8	编译	7
9	技能	6
10	生成	6

该文件的中文高频词（材料、报告、实验、编译、技能、生成）承担了项目的语义叙述功能，而英文高频词（skill、report、md、pdf）则承担了技术标识功能。这些英文词是技能名称、文件扩展名和编译目标格式的名称，属于无法或不应翻译为中文的技术标签。中文词叙述”是什么”，英文词标记”叫什么”，这种功能分工在热力图中表现为 README.md 列在中文词行上有持续的色彩，同时在部分英文词行上有独立的深色色块。相比之下，SKILL.md 虽然也包含技术术语，但其 159 行的行文中中文叙述占绝对主导，英文词仅作为 LaTeX 命令、命令名等技术字符串零散出现，不形成连续的高频模式。

6 结论

我们通过本次实验对 6 份 Markdown 技能文档实施了系统的词频统计分析，建立了覆盖词频、N-gram 词组频率、TF-IDF 关键词和 Zipf 定律验证的多维度分析框架。

该语料的词频分布在整体上服从 Zipf 定律的幂律衰减，拟合斜率为 -0.879 ，中段数据（约第 10 至第 400 位）呈现稳定的幂律行为。斜率偏离理想值 -1.0 的原因

是语料中所有高频词均为内容词而非功能词，内容词的频次受语义覆盖密度的天然限制，无法拉开通用语料中功能词层级间的那种数量级差距。累积覆盖率分析表明，前 100 个高频词覆盖了语料 42.9% 的内容，覆盖率爬升缓慢的主要原因是排名第 50 至第 300 位之间存在大量中频词构成的“词汇高原”，这是规范性文档被拆分为大量子条款后产生的结构性特征。

语料的内部结构呈现出三个层次：SKILL.md 作为核心文件在几乎所有主题词上贡献最大，确立了语料的主题统一性；README.md 以中文叙述搭配英文技术标签的模式构成了独立的信息层次；四份参考文件则各自以其局部高频词反映了不同的功能侧重点：final-review.md 侧重删除与审阅，evidence-grounding.md 侧重证据与声称，source-recovery.md 侧重代码命令模板。N-gram 分析进一步揭示了规范性文档的文体特征：「而非」作为排除性修辞标记以 26 次的最高频次出现在 bigram 中，折射出技能文件以否定式约束密集驱动的写作风格。

我们建立的文本分析流水线仅依赖 Python 标准库和少量第三方包即可完成从原始 Markdown 文档到多维统计图表的全流程分析，对 6 份文档共 5,719 个有效 token 的语料表现出了充分的统计敏感性，能够识别词频分布中的结构特征、文件间的主题差异和 N-gram 层面的文体规律。